

第 11 章 软件需求工程

软件需求工程是包括创建和维护软件需求文档所必需的一切活动的过程，可分为需求开发和需求管理两大工作。需求开发包括需求获取、需求分析、编写需求规格说明书（需求定义）和需求验证 4 个阶段。在需求开发阶段需要确定软件所期望的用户类型，获取每种用户类型的需求，了解实际的用户任务和目标，以及这些任务所支持的业务需求。同时还包括分析源于用户的信息，对需求进行优先级分类，将所收集的需求编写成需求规格说明书和需求分析模型，以及对需求进行评审等工作。需求管理通常包括定义需求基线、处理需求变更和需求跟踪等方面的工作。

11.1 软件需求概述

软件需求是指用户对新系统在功能、行为、性能、设计约束等方面的期望。根据 IEEE 的软件工程标准词汇表，软件需求是指用户解决问题或达到目标所需的条件或能力，是系统或系统部件要满足合同、标准、规范或其他正式规定文档所需具有的条件或能力，以及反映这些条件或能力的文档说明。

11.1.1 需求的层次

简单地说，软件需求就是系统必须完成的事以及必须具备的品质。需求是多层次的，包括业务需求、用户需求和系统需求，这三个不同层次从目标到具体，从整体到局部，从概念到细节。

1. 业务需求

业务需求是指反映企业或客户对系统高层次的目标要求，通常来自项目投资人、购买产品的客户、客户单位的管理人员、市场营销部门或产品策划部门等。通过业务需求可以确定项目视图和范围，项目视图和范围文档把业务需求集中在一个简单、紧凑的文档中，该文档为以后的开发工作奠定了基础。

2. 用户需求

用户需求描述的是用户的具体目标，或用户要求系统必须能完成的任务。也就是说，用户需求描述了用户能使用系统来做些什么。通常采取用户访谈和问卷调查等方式，对用户使用的场景（scenarios）进行整理，从而建立用户需求。

3. 系统需求

系统需求是从系统的角度来说明软件的需求，包括功能需求、非功能需求和设计约束等。功能需求也称为行为需求，它规定了开发人员必须在系统中实现的软件功能，用户利用这些功

能来完成任务，满足业务需要。功能需求通常是通过系统特性的描述表现出来的，所谓特性，是指一组逻辑上相关的功能需求，表示系统为用户提供某项功能（服务），使用户的业务目标得以满足；非功能需求是指系统必须具备的属性或品质，又可细分为软件质量属性（例如，可维护性、效率等）和其他非功能需求。

11.1.2 质量功能部署

质量功能部署（Quality Function Deployment, QFD）是一种将用户要求转化成软件需求的技术，其目的是最大限度地提升软件工程过程中用户的满意度。为了达到这个目标，QFD 将软件需求分为三类：常规需求、期望需求和意外需求。

（1）常规需求。用户认为系统应该做到的功能或性能，实现越多用户会越满意。

（2）期望需求。用户想当然认为系统应具备的功能或性能，但并不能正确描述自己想要得到的这些功能或性能需求。如果期望需求没有得到实现，会让用户感到不满意。

（3）意外需求。意外需求也称为兴奋需求，是用户要求范围外的功能或性能（但通常是软件开发人员很乐意赋予系统的技术特性），实现这些需求用户会更高兴，但不实现也不影响其购买的决策。意外需求是控制在开发人员手中的，开发人员可以选择实现更多的意外需求，以便得到高满意、高忠诚度的用户，也可以（出于成本或项目周期的考虑）选择不实现任何意外需求。

11.1.3 软件需求

软件需求目前并没有统一的定义，但都包含以下几方面的内容：

（1）用户解决问题或达到目标所需条件或权能。

（2）系统或系统部件要满足合同、标准、规范或其他正式规定文档所需具有的条件或权能。

（3）一种反映上面（1）或（2）所述条件或权能的文档说明，即需求规格说明。它包括功能性需求及非功能性需求，非功能性需求对设计和实现提出了限制，比如性能要求、质量标准或者设计限制。

在经典的瀑布软件过程模型中，将需求分析作为软件开发的第一个阶段，明确指出该阶段的输出成果为用户原始需求说明书和软件需求规格说明。从这点看，需求阶段首先需要定义用户的原始需求，并与用户、客户达成一致；其次，需要对原始需求进行分析，给出一个初步的软件解决方案，并给出该软件的需求规格说明，以指导后续的软件开发。这两个文档之间存在一个转换过程。

在统一过程（UP）中，需求按照“FURPS+”模型进行分类。“FURPS+”中的“FURPS”具体指以下需求：

（1）功能性（Functional）：特性、功能、安全性。

（2）可用性（Usability）：人性化因素、帮助、文档。

（3）可靠性（Reliability）：故障频率、可恢复性、可预测性。

（4）性能（Performance）：响应时间吞吐量、准确性、有效性、资源利用率。

（5）可支持（Sptability）：适应性、可维护性、国际化、可配置性。

“FURPS+”中的“+”是指一些辅助性的和次要的因素，用来强调各种不同的属性。比如：

- (1) 实现 (Implementation): 资源限制、语言和工具、硬件等。
- (2) 接口 (Interface): 强加于外部系统接口之上的约束。
- (3) 操作 (Operation): 对其操作设置的系统管理。
- (4) 包装 (Packaging): 例如物理的包装盒。
- (5) 授权 (Legal): 许可证或其他方式。

IEEE 在软件需求规格说明标准中, 把好需求定义为正确、无歧义、可检验和可跟踪的需求; 把好的需求集合定义为完整、一致和可修改的需求集合; 把高品质软件需求特性定义为正确性、完整性、一致性、明确性、可验证性、依照重要程度及稳定度进行优先排序、可修改性、可追溯性、可理解性。

11.1.4 需求工程

需求工程是随着计算机的发展而发展的, 在计算机发展的初期, 软件规模不大, 软件开发所关注的是代码编写, 需求分析很少受到重视。随着软件系统规模的扩大, 需求分析与定义在整个软件开发与维护过程中越来越重要, 直接关系到软件的成功与否。人们逐渐认识到需求分析活动不再仅限于软件开发的最初阶段, 它贯穿于系统开发的整个生命周期。需求工程是指应用已证实有效的原理、方法, 通过合适的工具和记号, 系统地描述待开发系统及其行为特征和相关约束。需求工程覆盖了体系结构设计之前的各项开发活动, 主要包括分析客户要求、对未来系统的各项功能性及非功能性需求进行规格说明。需求工程的目标简单明了: 确定客户需求, 定义设想中系统的所有外部特征。需求工程的活动主要被划分为以下几个阶段:

(1) 需求获取。通过与用户的交流, 对现有系统的观察及对任务进行分析, 从而开发、捕获和修订用户的需求。

(2) 需求分析。为系统建立一个概念模型, 作为对需求的抽象描述, 并尽可能多地捕获现实世界的语义。

(3) 形成需求规格(或称为需求文档化)。按照相关标准, 生成需求模型的文档描述, 用户的原始需求说明书作为用户和开发者之间的一个协约, 往往被作为合同的附件; 软件需求描述规约作为后续软件系统开发的指南。

(4) 需求确认与验证。以需求规格说明为输入, 通过用户确认、复审会议、符号执行、模拟仿真或快速原型等途径与方法, 确认和验证需求规格的完整性、正确性、一致性、可测试性和可行性, 包含有效性检查、一致性检查、可行性检查和确认可验证性。

(5) 需求管理。需求管理包括需求文档的追踪管理、变更控制、版本控制等管理性活动。软件需求开发的最终文档经过评审批准后, 就定义了开发工作的需求基线(Baseline)。这个基线在客户和开发者之间构筑了计划产品功能需求和非功能需求的一个约定(Agreement)。需求约定是需求开发和需求管理之间的桥梁。

11.2 需求获取

需求获取是一个确定和理解不同的项目干系人的需求和约束的过程。需求获取是一件看上

去很简单，做起来却很难的事情。需求获取是否科学、准确、充分，对获取的结果影响很大，这是因为大部分用户无法完整地描述需求，而且也不可能看到系统的全貌。因此，需求获取只有通过系统分析师与用户的有效合作才能成功。系统分析师必须建立一个对问题进行彻底探讨的环境，而这些问题与将要开发的系统有关。让用户明确了解，对于某些功能的讨论并不意味着即将在系统中实现它。

在需求获取的过程中，将会产生大量的信息，系统分析师要将这些信息有条理地记录下来，形成用户原始需求说明书文档。

11.2.1 用户访谈

用户访谈是最基本的一种需求获取手段，其形式包括结构化和非结构化两种。结构化是指事先准备好一系列问题，有针对地进行；非结构化则是只列出一个粗略的想法，根据访谈的具体情况发挥。最有效的访谈是结合这两种方法进行，毕竟不可能把什么都一一计划清楚，应该保持良好的灵活性。

为了进行有效的用户访谈，系统分析师需要在三个方面进行组织，分别是准备访谈、主持访谈和访谈的后续工作。

准备访谈过程中，首先也是最重要的步骤是确定访谈的目的，其次是确定访谈中应该包括哪些用户。这两个步骤结合得非常紧密，因此通常一起完成。参加访谈的用户人数取决于访谈的目的。通常，最好限制参加访谈的人数。准备访谈的第三步是为访谈准备一些详细的问题。系统分析师可以根据已经获得的表格和报表写出一些具体的问题，并作好笔记。问题可以分为开放式问题和封闭式问题两类。准备访谈的最后是作出最终的访谈安排，并把这些安排通知所有参加者。

在访谈的过程中，要做好以下几项工作：限制访谈时间、寻找异常和错误情况、深入调查细节，同时需要认真作好记录。在访谈时，系统分析师一定要注意措辞得当，充分尊重用户；注意保持轻松的气氛，在说话、提问时应该尽量采用易于理解和通俗化的语言，避免使用 IT 专业术语。

访谈后续工作的首要任务是吸收、理解和记录访谈所获得的信息。通常，系统分析师通过构造业务过程的模型来记录访谈的细节，和项目组其他成员一起复查访谈中发现的结果，然后在一天或至多两天内记录下结果。

总的来说，用户访谈具有良好的灵活性，应用范围比较广泛。但是，它也存在许多困难，例如，用户经常较忙，难以安排时间；面谈时信息量大，记录较为困难；沟通需要很多技巧，同时需要系统分析师具有足够的领域知识等。另外，在访谈时，还可能会遇到一些对于企业来说比较机密和敏感的话题。因此，这个看似简单的技术，也需要系统分析师具有丰富的经验和较强的沟通能力。

11.2.2 问卷调查

用户访谈最大的难处在于很多关键人员时间有限，不容易安排过多的时间。而且，如果用户较多，不可能一一访谈。因此，就需要借助问卷调查，通过精心设计调查表，然后下发到相

关的人员手中,让他们填写答案。这样,就可以有效地克服用户访谈方法中存在的问题。

问卷调查表使系统分析师可以从大量的项目干系人处收集信息,甚至当项目干系人在地理上分布很广时,他们仍然能通过问卷调查表来帮助获取需求。一张好的问卷调查表要花费大量的时间来进行设计与制作,包括确定问题及其类型、编写问题、设计问卷调查表的格式三个重要活动。

与用户访谈相比,问卷调查可以在短时间内,以低廉的代价从大量的回答中收集数据;问卷调查允许回答者匿名填写,大多数用户可能会提供真实信息;问卷调查的结果比较好整理和统计。

问卷调查最大的不足就是缺乏灵活性,因此,较好的做法是将用户访谈和问卷调查结合使用。具体来说,就是先设计问题,制作成问卷调查表,下发填写完后,进行仔细的分组、整理和分析,以获得基础信息。然后,再针对分析的结果进行小范围的用户访谈,作为补充。

11.2.3 采样

采样是指从种群中系统地选出有代表性的样本集的过程,通过认真研究所选出的样本集,可以从整体上揭示种群的有用信息。对于信息系统的开发而言,现有系统的文档(文件)就是采样种群。当开始对一个系统做需求分析时,查看现有系统的文档是对系统有初步了解的最好方法。但是,系统分析师应该查看哪些类型的文档?当文档的数据庞大,无法一一研究时,就需要使用采样技术选出有代表性的数据。

在种群中选择样本的技术主要有简单随机采样、分层采样、聚类采样、系统采样等,具体采取哪种选择方法,需要根据系统文档的数量、样本大小和实际情况进行决定。

采样技术不仅可以用于收集数据,还可以用于采集访谈用户或者是采集观察用户。在对人员进行采样时,上面介绍的采样技术同样适用。通过采样技术,选择部分而不是种群的全部,不仅加快了数据收集的过程,而且提高了效率,从而降低了开发成本。另外,采样技术使用了数理统计原理,能减少数据收集的偏差。

但是,由于采样技术基于统计学原理,样本规模的确定依赖于期望的可信度和已有的先验知识,很大程度上取决于系统分析师的主观因素,对系统分析师个人的经验和能力依赖性很强,要求系统分析师具有较高的水平和丰富的经验。

11.2.4 情节串联板

在需求获取的过程中,虽然系统分析师很大的一部分精力在于理解和分析业务,了解潜在的问题,但仍然不可避免地涉及一些解决方案的探讨,因为只有让用户了解“系统如何做”时才会更容易达成共识。而且,很多用户对信息系统是没有直观认识的,这样就很容易产生盲区,而这种时候,系统分析师就需要通过情节串联板技术来帮助用户消除盲区,达成共识。

1. 情节串联板的概念

情节串联板通常就是一系列图片,系统分析师通过这些图片来讲故事。在一般情况下,图片的顺序与活动事件的顺序一致,通过一系列图片说明会发生什么。人们发现,通过以图片辅

助讲故事的方式叙述需求，有助于有效和准确地沟通。在情节串联板中可以使用的图片类型包括流程图、交互图、报表和记录结构等。

简单地说，情节串联板技术就是使用工具向用户说明（或演示）系统如何适合企业的需要，并表明系统将如何运转。系统分析师将初始的情节串联板展示给讨论小组，小组成员提供意见。

2. 情节串联板的类型

情节串联板的类型包括被动式、主动式和交互式，其复杂程度依次递增。

被动式情节串联板通常由草图、图片、屏幕截图、幻灯片等组成。系统分析师充当系统的角色，让用户预演情节串联板，简单地表述为“当这样做时，会出现这样的情景”。

主动式情节串联板试图使用户能够看到类似“电影样片”的内容，它可以自动播放，描述系统在典型用法或典型场景中的行为方式。

交互式情节串联板让用户体验系统的行为，系统需要用户的参与才能继续运行。交互式情节串联板可以是仿真器、实物模型，甚至是抛弃式原型。

3. 情节串联板的优缺点

由于情节串联板给用户一个直观的演示，因此它是最生动的需求获取技术，其优点是用户友好、交互性强，对用户界面提供了早期的评审。情节串联板的缺点是花费的时间很多，使需求获取的速度大大降低。

11.2.5 联合需求计划

为了提高需求获取的效率，越来越多的企业倾向于使用小组工作会议来代替大量独立的访谈。联合需求计划（Joint Requirement Planning, JRP）是一个通过高度组织的群体会议来分析企业内的问题并获取需求的过程，它是联合应用开发（Joint Application Development, JAD）的一部分。

1. 联合应用开发

JAD 是以小组形式定义和建立系统的，它是由企业主管部门经理、会议主持人、用户、协调人员、IT 人员、秘书等共同组成的专题讨论组。由这个专题讨论组来定义并详细说明系统的需求和可选的技术方案。JAD 的过程大致如下：

- （1）确定 JAD 项目，主要指确定系统的范围和规范。
- （2）在 JAD 专题预备会上，会议主持人向参与者介绍项目和 JAD 专题讨论内容。
- （3）准备 JAD 专题讨论材料。
- （4）进行 JAD 专题讨论会，其目的是要达成对需求的一致意见，并对各种可选的技术方案加以讨论，从中研究出几套可供选择的方案。

JAD 方法充分发挥了 JAD 专题讨论会的优势，以便更好地满足用户的需求。使用 JAD 方法，比传统的收集需求的时间更短，可以加速系统开发周期。JAD 方法充分发挥了管理人员和用户的积极性，增强了管理人员和用户的责任感，从而使系统开发工作做得更好。

2. JRP 会议

联合需求计划（Joint Requirement Planning, JRP）是一种相对来说成本较高的需求获取方法，但也是一种十分有效的方法。它通过联合各个关键用户代表、系统分析师、开发团队代表，通过有组织的会议来讨论需求。通常该会议的参与人数为6~18人，召开时间为1~5h。

在会议之前，应该将与讨论主题相关的材料提前分发给所有将要参加会议的人。在会议开始之后，按照以下步骤进行：

（1）应该花一些时间让所有的与会者互相认识，以使交流在更加轻松的气氛下进行。会议开始时，针对所列举的问题进行逐项专题讨论。

（2）对现有系统和类似系统的不足进行开放性交流。鼓励与会者在短时间内说出尽量多的想法，在这一过程中不对这些想法发表任何评论。

（3）大家在此基础上对新的解决方案进行一番设想，在这个过程中，需要把这些想法、问题、不足记录下来，形成一个要点清单。

（4）针对这个要点清单进行整理，明确优先级，并进行评审。

为了更好地进行以后可能碰到的类似 JRP 会议，JRP 会议后一般会让与会者完成一个评价性的调查问卷。JRP 会议最后有一个总结性的报告，主要内容是与与会者达成一致的需求和未解决的问题。

3. 主要原则

JRP 的主要意图是收集需求，而不是对需求进行分析和验证。实施 JRP 时应把握以下主要原则：

- （1）在 JRP 实施之前，应制定详细的议程，并严格遵照议程进行。
- （2）按照既定的时间安排进行。
- （3）尽量完整地记录会议期间的内容。
- （4）在讨论期间尽量避免使用专业术语。
- （5）充分运用解决冲突的技能。
- （6）会议期间应设置充分的间歇时间。
- （7）鼓励团队取得一致意见。
- （8）保证参加 JRP 的所有人员能够遵守事先约定的规则。

JRP 将会起到群策群力的效果，对于一些最有歧义的问题、需求最不清晰的领域都是十分有用的一种方法。这种方法最大的难度是会议的组织和相关人员的能力，要做到言之有物，气氛开放，否则，将难以达到预想的效果。

11.2.6 需求记录技术

在需求获取的过程中，将会产生大量的信息，系统分析师要将这些信息有条理地记录下来，就需要借助一些工具。在信息系统开发实践中，有时候进行需求获取的人员和进行需求分析的人员不是同一个人（团队），有时候在同一个项目中有多个系统分析师参加需求获取，因此，需要统一需求记录工具，以便让所有人的获取结果是同一口径的。

常用的需求记录工具有任务卡片、场景说明、用户故事和 Volere 白卡等。

1. 任务卡片

在各种需求记录工具中，任务卡片是一种比较简单的工具，它特别适合对业务活动级的信息收集与整理。常用的任务卡片示例如图 11-1 所示。

任务：开通课程 目的：为用户开通权限，将其标记为“学员” 触发：/ 前提：用户交费到账 频率：平均每天880人次 关键情况：单位集体报名
子任务： 1. 查找用户资料 2. 将用户标记为“学员” 3. 为用户增加学习币
任务变体： 1a. 用户已经是学员 1b. 用户所交费用多于所选课程的规定费用 2a. 老学员

图 11-1 任务卡片示例

在图 11-1 中，各个项目的内容及解释如表 11-1 所示。

表 11-1 任务卡片的内容与要点

项 目	内 容	说 明
任务	对该业务活动进行命名	一定要使用用户的业务术语
目的	以业务活动的工作意义进行概述	说明的是意图而非动作
触发	进行该业务活动	系统要判断的前置条件
前提	触发该业务活动的时机和场景	说明了业务前提
频率	任务发生的频率	这是一种非功能需求
关键情况	一些十分特殊的业务场景	系统需要专门进行处理
子任务	该业务活动的具体步骤	相当于用例的基本事件流
任务变体	该业务活动的变体与异常处理	相当于用例的扩展事件流

任务卡片还有一个增强版，它的信息更加全面，如图 11-2 所示。

增强版任务卡片在基本任务卡片的基础上，增加了问题点描述和解决方案提示。其中方案示例是针对问题点，系统需要实现什么样的功能，以便验证这些解决方案是否能够解决用户提出的问题。

任务：开通课程 目的：为用户开通权限，将其标记为“学员” 触发：/ 前提：用户交费到账 频率：平均每天880人次 关键情况：单位集体报名	
子任务： 3.为用户增加学习币 问题：(1)用户同时选修多门课程 (2)用户可能要求多加学习币	方案示例： 多门课程的选修采取权限控制的方式 系统显示各门课程的学习币情况
.....	
任务变体： 1b.用户所交费用多于所选课程的规定费用 问题：用户没有明确多余的费用用途	系统自动将多余的费用转换为用户 的学习币
.....	

图 11-2 增强版任务卡片示例

2. 场景说明

有时候，系统分析师可能很难总结出子任务和任务变体，因为这需要对任务执行过程进行抽象。此时，系统分析师可以使用场景说明来对用户的描述进行整理，抽象出子任务。简单地理解，场景说明就是用户对其工作场景和过程的详细描述，这些描述将在编写测试用例和用户培训手册中再次用到。

3. 用户故事

用户故事描述了对用户有价值的功能，可包括三个方面内容，分别是书面描述（用于计划和备忘）、交谈（细化故事）和测试用例（验证故事实现）。用户故事描述的传统形式是手工写的用户故事卡，系统分析师辅助用户编写故事，告诉用户所编写的故事是进一步讨论的引子，而不是详细的需求规范。在任何项目中，需要用户团队根据故事的重要性来安排开发工作，回答所有开发问题，编写所有的故事。在编写故事之前应该建立用户角色模型，必须包含对项目成功至关重要的角色，尽量保证所有用户对系统完全满意。

用户故事具有6个基本属性：独立性、可协商性、对用户有价值、可预测性、短小精悍和可测试性。

(1) 独立性。尽可能避免故事之间存在依赖关系，因为依赖关系会产生优先级和规划问题。

(2) 可协商性。故事是可协商的，不是必须实现的书面合同或者需求。

(3) 对用户有价值。确保每个故事对用户有价值的最好方式是让用户编写故事。

(4) 可预测性。系统分析师应该能够预测（至少大致猜测）故事的规模，以及实现所需要的工作量。

(5) 短小精悍。故事规模对实现有影响，哪种故事规模最合适，取决于开发团队的规模和能力，以及技术实现等方面。

(6) 可测试性。所编写的故事必须是可测试的。

4. Volere 白卡

Volere 白卡是一种类似于任务卡片的需求记录工具，其格式如图 11-3 所示。

需求编号:	需求类型:	事件/用例编号:
描述:		
理由:		
来源:		
验收标准:		
顾客满意度:	顾客不满意度:	
依赖关系:	冲突:	
支持材料:		
历史:		

图 11-3 Volere 白卡的格式

用户故事和 Volere 白卡定位的是最小的需求项，因此在实际应用中会导致量比较大，一般在敏捷方法中使用。

系统分析师在选择需求记录工具时，既可以借鉴现有的模板，也可以根据自己的需要进行扩展或重新定义。另外，选择记录工具时要考虑项目团队所使用的开发方法、用户的实际情况、系统分析师的技能等因素。

11.3 需求分析

在需求获取阶段，系统分析师所获得的需求是杂乱的，是用户对新系统的期望和要求，这些要求有重复的地方，也有矛盾的地方，这样的要求是不能作为软件设计的基础的。一个好的需求应该具有无二义性、完整性、一致性、可测试性、确定性、可跟踪性、正确性、必要性等特性，因此，需要系统分析师把杂乱无章的用户要求和期望转化为用户需求，并通过多种具体的需求分析方法和工具，将用户需求转化为用于指导后续软件设计和开发的软件需求描述规约，这就是需求分析的工作。

11.3.1 需求分析的任务

需求分析的目标是把用户对待开发软件提出的“要求”或“需要”进行分析与整理，确认后形成描述完整、清晰与规范的文档，确定软件需要实现哪些功能，完成哪些工作。此外，软件的一些非功能性需求（如软件性能、可靠性、响应时间、可扩展性等），软件设计的约束条件，运行时与其他软件的关系等也是软件需求分析的目标。

11.3.2 需求分析的方法

在软件工程实践过程中，人们提出了许多种需求分析的方法，其中主要有结构化分析

(Structured Analysis, SA)方法、面向对象的分析(Object-Oriented Analysis, OOA)方法和面向问题域的分析(Problem Domain Oriented Analysis, PDOA)方法。另外,还有一些形式化方法,例如,维也纳设计方法(Vienna Design Method, VDM)。本节只简单介绍PDOA方法。

1. PDOA 方法

与SA方法和OOA方法相比,PDOA方法更多地强调描述,而较少强调建模。它的描述大致分为以下两个部分:

(1) 关注问题域。用一个文档对含有的问题域进行相关的描述,并列出需要在该域中求解的问题列表,也就是需求列表。只有这个文档是在分析时产生的。

(2) 关注解系统(即系统实现)的待求行为。用一个文档对解系统的待求行为进行描述。该文档将在需求定义阶段完成。

在PDOA方法中,对整个过程有着清晰的定义:

(1) 收集基本的信息并开发问题框架,以建立问题域的类型。

(2) 在问题框架类型的指导下,进一步收集详细信息,并给出一个问题域相关特性的描述。

(3) 基于以上两点,收集并用文档说明新系统的需求。

从上面的描述中可以看出,问题框架是PDOA的核心元素,是将问题域分为一系列相互关联的子域,而一个子域可以是那些可能算是精选出来的问题域的一部分。也可以把问题框架看作是开发上下文范围关系图,但不同的是,上下文范围关系图的建模对象是针对解系统,而问题框架则是针对问题域。也就是说,问题框架的目标就是大量地获取更多有关问题域的信息。

2. 方法的对比

SA方法关注于功能的分层和分解,这非常符合人们自上而下、逐步分解问题直到可解决的思考方式。SA方法本身隐含着几个基本假设,即问题域是可定义的、问题域是有限的、通过有限的步骤总可以将复杂问题分解到可解决的程度。SA方法应用的是科学方法中的因果律、归纳法和逻辑法,通过对现实世界中的问题域进行不断的“测量”和“分解”,直到得到问题域的逻辑模型。

OOA方法则遵循完全不同的思维方式,它基于抽象、信息隐藏、功能独立和模块化这些基本理念对系统进行分析。OOA方法首先对问题域的事物的“外在表象”进行观测,然后在逻辑世界中模拟出一个对应的逻辑对象,“断定”该对象和现实事物是一致的。随后,观测到的对象被记录入对象集合,观测到的行为和表象被记录入对象关系模型和对象行为模型。OOA方法建立的对象彼此之间通过接口来相互沟通,每传递一个消息即触发一个事件,并引起内部方法的执行。只有观测对象内部的时候,才能看到具体的属性和方法。否则,只能看到对象对外部开放的接口。

SA方法假定系统分析师理解问题域的全部,并且有能力正确地识别和分解问题。而OOA方法既不假定系统分析师理解问题域的全部,也不假定其能够建立正确的抽象对象,它只承诺一种可以持续“观测并理解”的方法,以及“观测后建立”的对象和现实世界的外在表象是一致的。

很难对SA方法和OOA方法做优劣性的比较,使用两种方法成功和失败的软件系统都很

多。OOA 方法已经成为当前的主流分析方法，拥有大量的语言和建模工具的支持。然而，SA 方法也并未过时，很多成功的软件系统依然在通过 SA 方法进行分析和实现。

PDOA 方法的特点是重新将重点定位在问题域和需求上，通过对问题域的分类，向系统分析师提供具体问题的相关指南，并且它将规格说明作为另外的任务处理，它的成果只是一份问题域的全面描述和一份需求列表而已。PDOA 方法丰富和完善了 SA 方法和 OOA 方法，然而人们对它的了解和掌握还比较有限。

11.4 结构化分析方法

SA 方法的基本思想是自顶向下，逐层分解，把一个大问题分解成若干个小问题，每个小问题再分解成若干个更小的问题。经过逐层分解，每个最低层的问题都是足够简单、容易解决的，于是复杂的问题也就迎刃而解了。在 SA 方法中导出的分析模型如图 11-4 所示。

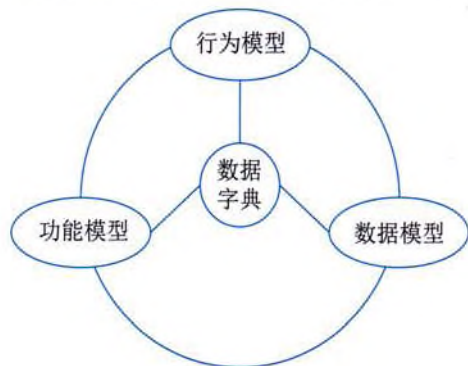


图 11-4 需求分析模型

从图 11-4 可以看出，SA 方法分析模型的核心是数据字典，围绕这个核心，有三个层次的模型，分别是数据模型、功能模型和行为模型（也称为状态模型）。在实际工作中，一般使用 E-R 图表示数据模型，用 DFD 表示功能模型，用状态转换图（State Transform Diagram, STD）表示行为模型。这三个模型有着密切的关系，它们的建立不具有严格的时序性，而是一个迭代的过程。

11.4.1 数据流图

DFD 是 SA 方法中的重要工具，是表达系统内数据的流动并通过数据流描述系统功能的一种方法。DFD 还可被认为是一个系统模型，在信息系统开发中，如果采用结构化方法，则一般将 DFD 作为需求规格说明书的一个组成部分。

1. DFD 的主要作用

DFD 从数据传递和加工的角度，利用图形符号通过逐层细分描述系统内各个部件的功能和数据在它们之间传递的情况，来说明系统所完成的功能。具体来说，DFD 的主要作用如下：

(1) DFD 是理解和表达用户需求的工具，是需求分析的手段。由于 DFD 简明易懂，不需要任何计算机专业知识就可以理解它，因此，系统分析师可以通过 DFD 与用户进行交流。

(2) DFD 概括地描述了系统的内部逻辑过程，是需求分析结果的表达工具，也是系统设计的重要参考资料，是系统设计的起点。

(3) DFD 作为一个存档的文字材料，是进一步修改和充实开发计划的依据。

2. DFD 的基本符号

在 DFD 中，通常会出现 4 种基本符号，分别是数据流、加工、数据存储和外部实体（数据

源及数据终点)。数据流是具有名字和流向的数据,在DFD中用标有名字的箭头表示。加工是对数据流的变换,一般用圆圈表示。数据存储是可访问的存储信息,一般用直线段表示。外部实体是位于被建模的系统之外的信息生产者或消费者,是不能由计算机处理的成分,它们分别表明数据处理过程的数据来源及数据去向,用标有名字的方框表示。

3. DFD 的层次

SA方法的思路是依赖于DFD进行自顶向下的分析。这也是因为系统通常比较复杂,很难在一张图上就将所有的数据流和加工描述清楚。因此,DFD提供一种表现系统高层和低层概念的机制。也就是先绘制一张较高层次的DFD,然后在此基础上,对其中的加工进行分解,分解成为若干个独立的、低层次的、详细的DFD,而且可以这样逐一分解下去,直至系统被清晰地描述出来。

(1) 顶层图。顶层图是描述系统最高层结构的DFD,它的特点是将整个待开发的系统表示为一个加工,将所有的外部实体和进出系统的数据流都画在一张图中。例如,图11-5就是一个顶层图的实例,只不过在绘制时做了一些处理,使得它看上去更加直观易懂。

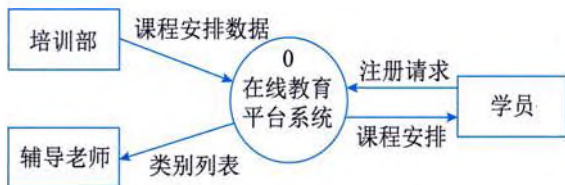


图 11-5 顶层图示例

顶层图用来描述系统有什么输入和输出数据流,与哪些外部实体直接相关,可以把整个系统的范围勾画出来。

(2) 逐层分解。当完成了顶层图的建模之后,就可以在此基础上开展进一步的分解。对图11-5进行分解,在对原有流程了解的基础上,可以得到图11-6。

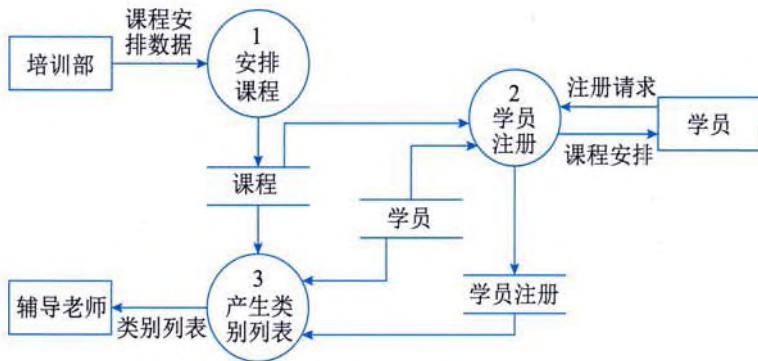


图 11-6 0层图示例

图11-6是在顶层图11-5的基础上做的第一次分解,而在图11-6中只有一个加工,那就是系统本身,可以将其编号为0。因此,对顶层图进行的分解,其实就是对这个编号为0的加工

进行更细化的描述，在这里引入了新的加工和数据存储，为了能够区分其所在的级别，在这个层次上的加工将以 1, 2, 3, ... 为序列进行编号。

正是由于这是对加工 0 的分解，因此也称为 0 层图。可以根据需要对 0 层图上的加工进行类似的再分解，称之为 1 层图。在 1 层图中引入的新加工，其编号规则就是 1.1, 1.2, 1.3, ...，以及 2.1, 2.2, 2.3, ...。以此类推，直到完成分析工作。

4. 如何画 DFD

DFD 的绘制是一个自顶向下、由外到里的过程，通常按照以下几个步骤进行：

(1) 画系统的输入和输出：在图的边缘标出系统的输入数据流和输出数据流。这一步其实是决定研究的内容和系统的范围。在画的时候，可以先将尽可能多的数据流画出来，然后再删除多余的，增加遗漏的。

(2) 画 DFD 的内部：将系统的输入、输出用一系列的处理连接起来，可以从输入数据流画向输出数据流，也可以从中间画出去。

(3) 为每一个数据流命名：命名的好坏与 DFD 的可理解性密切相关，应避免使用空洞的名字。

(4) 为加工命名：使用动宾短语为每个加工命名。

每画好一张 DFD，就需要进行检查和修改，检查和修改的原则如下：

(1) DFD 中的所有图形符号只限于前述 4 种基本图形元素，图上每个元素都必须有名字。

(2) 每个加工至少有一个输入数据流和一个输出数据流，而且要保持数据守恒。也就是，一个加工的所有输出数据流中的数据必须能从该加工的输入流中直接获得，或者通过该加工能产生的数据。一个加工的输出数据流不应与输入数据流同名，即使它们的组成完全相同。

(3) 在 DFD 中，须按层给加工编号。编号表明了该加工处在哪一层，以及上下层的父图与子图的对应关系。

(4) 规定任何一个 DFD 子图必须与它上一层的一个加工对应，两者的输入数据流和输出数据流必须一致。这就是父图与子图的平衡。也就是说，父图中的某加工的输入 / 输出数据流必须与它的所有子图的输入 / 输出数据流在数量上和名字上相同。值得注意的是，如果父图中的一个输入 / 输出数据流对应于子图中的几个输入 / 输出数据流，而子图中组成这些数据流的数据项的全体正好是父图中的这一个数据流，那么它们仍然算是平衡的。

(5) 在整套 DFD 中，每个数据存储必须既有读的数据流，又有写的数据流。但是在某张子图中，可能只有读没有写，或者只有写没有读。

(6) 可以在 DFD 中加入物质流，帮助用户理解 DFD，但不可夹带控制流。

11.4.2 状态转换图

大多数业务系统是数据驱动的，所以适合使用 DFD。但是，实时控制系统却主要是事件驱动的，因此，行为模型是最有效的描述方式。STD 通过描述系统的状态和引起系统状态转换的事件，来表示系统的行为。此外，STD 还指出了作为特定事件的结果将执行哪些动作（例如，处理数据等）。

状态是任何可以被观察到的系统行为模式，每个状态代表系统的一种行为模式。在 STD 中，用圆形框或椭圆框表示状态，通常在框内标上状态名。状态规定了系统对事件的响应方式。系统对事件的响应可以是做一个（或一系列）动作，也可以是仅仅改变系统本身的状态。STD 描述了系统如何在各种状态之间移动。

事件是在某个特定时刻发生的事情，它是对引起系统从一个状态转换到另一个状态的外界事件的抽象。例如，内部时钟指明某个规定的时间段已经过去，鼠标移动或单击等都是事件。简而言之，事件就是引起系统状态转换的控制信息。

在 STD 中，从一个状态到另一个状态的转换用箭头线表示，箭头表明转换方向，箭头线上标上事件名。必要时可在事件名后面加上方括号，括号内写上状态转换的条件。也就是说，仅当方括号内所列出的条件为真时，该事件的发生才引起箭头所示的状态转换。图 11-7 给出了一个在线课程学习的 STD 示例。

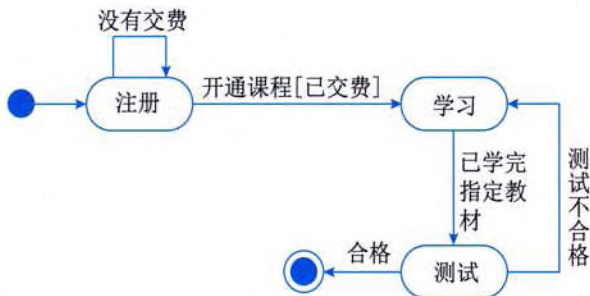


图 11-7 STD 示例

STD 既可以表示循环运行过程，也可以表示单程生命期。当描述循环运行过程时，通常不关心循环是怎样启动的。当描述单程生命期时，需要标明初始状态（简称为“初态”，系统启动时进入初始状态）和最终状态（简称为“终态”，系统运行结束时到达最终状态）。在 STD 中，初始状态用实心圆表示，最终状态用一对同心圆（内圆为实心圆）表示。

11.4.3 数据字典

DFD 描述了系统的分解，即系统由哪几部分组成，各部分之间的联系等，但是，对于数据的详细内容却无法在 DFD 中得到反映。例如，图 11-7 中的数据存储“课程”包括哪些内容，在 DFD 中就无法具体、准确地描述。数据字典是在 DFD 的基础上，对 DFD 中出现的所有命名元素都加以定义，使得每个图形元素的名字都有一个确切的解释。DFD 和数据字典等工具相配合，就可以从图形和文字两个方面对系统的逻辑模型进行完整的描述。

1. 数据字典的条目

数据字典中一般有 6 类条目，分别是数据元素、数据结构、数据流、数据存储、加工逻辑和外部实体。不同类型的条目有不同的属性需要描述。

(1) 数据元素。数据元素也称为数据项，是数据的最小组成单位，例如，课程号、课程名等。对数据元素的描述，应该包括数据元素的名称、编号、别名、类型、长度、取值范围和取

值的含义等。此外，数据元素的条目还包括对该元素的简要说明，以及与它有关的数据结构等信息。

(2) 数据结构。数据结构用于描述某些数据元素之间的关系，它是一个递归的概念，一个数据结构可以包括若干个数据元素和（或）数据结构。数据结构的描述重点是数据元素之间的组合关系，即说明数据结构包括哪些成分。这些成分中有三种特殊情况，分别是任选项、必选项和重复项。任选项是可以出现也可以省略的项，用“[]”表示。必选项是在两个或多个数据元素中，必须出现其中的一个。例如，任何一门课程是必修课或选修课，二者必居其一。必选项的表示是将候选的多个数据项用“{}”括起来。重复项是可以多次出现的数据项。例如，一张学员注册表可选择多门课程，“课程细节”可重复多次，表示成“课程细节*”。

(3) 数据流。数据流由一个或一组数据元素组成，对数据流的描述应包括数据流的名称、编号、简要说明、来源、去处、组成和流量（含高峰时期的流量）。

(4) 数据存储。数据存储的条目主要描写该数据存储的结构，以及有关的数据流和查询要求。有些数据存储的结构可能很复杂，例如，图 11-6 中的“学员”包括学员的基本情况、学员动态、在线模拟测试记录、论文练习成绩等，其中每一项又是一个数据结构，这些数据结构有各自的条目分别加以说明。因此，在“学员”的条目中只需列出这些数据结构，而不要列出数据结构的内部构成。DFD 是分层的，下层图是上层图的具体化。同一个数据存储可能在不同层次的图中出现。描述这样的数据存储，应列出最底层图中的数据流。

(5) 加工逻辑。加工逻辑需要描述加工的编号、名称、功能的简要说明、有关的输入数据流和输出数据流。对加工进行描述，目的在于使相关人员能有一个较明确的概念，了解加工的主要功能。详细的加工逻辑则需要借助一些工具来描述，包括判定树、判定表和结构化语言等。

(6) 外部实体。外部实体是数据的来源和去向，对外部实体的描述应包括外部实体的名称、编号、简要说明、外部实体产生的数据流和系统传给该外部实体的数据流，以及该外部实体的数量。外部实体的数量对于估计系统的业务量有参考作用，尤其是关系密切的主要外部实体。

2. 数据字典的作用

数据字典实际上是“关于系统数据的数据库”。在整个系统开发过程和系统运行与维护阶段，数据字典是必不可少的工具。数据字典是所有人员工作的依据，统一的标准。它可以确保数据在系统中的完整性和一致性。具体来讲，数据字典具有以下作用：

(1) 按各种要求列表。可以根据数据字典，把所有数据条目按一定的顺序全部列出，保证系统设计时不会遗漏。如果系统分析师要对某个数据存储的结构进行深入分析，需要了解有关的细节，了解数据结构的组成乃至每个数据元素的属性，数据字典也可提供相应的内容。

(2) 相互参照，便于系统修改。根据初步的 DFD，建立相应的数据字典。在需求分析过程中，系统分析师常会发现原来的 DFD 及各种数据定义中有错误或遗漏，需要修改或补充。有了数据字典，这种修改就变得容易多了。

(3) 由描述内容检索名称。在一个稍微复杂的系统中，系统分析师可能没有把握断定某个数据元素在数据字典中是否已经定义，或者记不清楚其确切名字时，可以由内容查找其名称。

(4) 一致性检验和完整性检验。根据各类条目的规定格式，可以发现一些问题，例如，是

否存在没有指明来源或去向的数据流, 是否存在没有指明数据存储或所属数据流的数据元素, 加工逻辑与输入的数据元素是否匹配, 是否存在没有输入或输出的数据存储等。

3. 数据字典的管理

为了保证数据的一致性, 数据字典必须由专人(数据管理员)管理。数据管理员的职责就是维护和管理数据字典, 保证数据字典内容的完整性和一致性。任何人, 包括系统分析师、软件设计师、程序员, 如果要修改数据字典的内容, 都必须通过数据管理员。数据管理员还要负责把数据字典的最新版本及时通知有关人员。

11.5 面向对象分析方法

OOA 的基本任务是运用 OO 方法, 对问题域进行分析和理解, 正确认识其中的事物及它们之间的关系, 找出描述问题域和系统功能所需的类和对象, 定义它们的属性和职责, 以及它们之间所形成的各种联系。最终产生一个符合用户需求, 并能直接反映问题域和系统功能的 OOA 模型及其详细说明。

OOA 模型独立于具体实现, 即不考虑与系统具体实现有关的因素, 这也是 OOA 和面向对象的设计(Object-Oriented Design, OOD)的区别所在。OOA 的任务是“做什么”, OOD 的任务是“怎么做”。

11.5.1 用例模型

从用户的角度来看, 他们并不想了解系统的内部结构和设计, 他们所关心的是系统所能提供的服务, 这就是用例方法的基本思想。用例方法是一种需求合成技术, 从获取到的要求和期望中进行整理与提炼, 从而建立用例模型。在 OOA 方法中, 构建用例模型一般需要经历 4 个阶段, 分别是识别参与者、合并需求获得用例、细化用例描述和调整用例模型, 其中前三个阶段是必需的。

1. 用例图的元素

用例是一种描述系统需求的方法, 使用用例的方法来描述系统需求的过程就是用例建模。在用例图中, 主要包括参与者、用例和通信关联三种元素, 如图 11-8 所示。

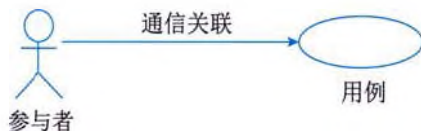


图 11-8 用例图中的基本元素

(1) 参与者。参与者是指存在于系统外部并与系统进行交互的任何事物, 既可以是使用系统的用户, 也可以是其他外部系统和设备等外部实体。

(2) 用例。用例是在系统中执行的一系列动作, 这些动作将生成特定参与者可见的价值结果。也就是说, 用例表示系统所提供的服务, 它定义了系统是如何被参与者所使用的, 它描述的是参与者为了使用系统所提供的某一完整功能而与系统之间发生的一段对话。

(3) 通信关联。通信关联表示的是参与者和用例之间的关系, 或用例与用例之间的关系。箭头表示在这一关系中哪一方是对话的主动发起者, 箭头所指方是对话的被动接受者, 箭尾所

指方是对话的主动发起者。如果不想强调对话中的主动与被动关系，可以使用不带箭头的关联实线。在用例模型中，信息流不是由通信关联来表示的。该信息流是默认存在的，并且是双向的，它与箭头所指的方向没有关系。

2. 识别参与者

参与者是与系统交互的所有事物，该角色不仅可以由人承担，还可以是其他系统和硬件设备，甚至是系统时钟。

(1) 其他系统。当系统需要与其他系统交互时，其他系统就是一个参与者。例如，对某企业的在线教育平台系统而言，该企业的 OA 系统就是一个参与者。

(2) 硬件设备。如果系统需要与硬件设备交互，硬件设备就是一个参与者。例如，在开发集成电路（Integrated Circuit, IC）卡门禁系统时，IC 卡读写器就是一个参与者。

(3) 时钟。当系统需要定时触发时，时钟就是一个参与者。例如，开发在线测试系统中的“定时交卷”功能时，就需要引入时钟作为参与者。

3. 合并需求获得用例

将参与者都找到之后，接下来就是仔细地检查参与者，为每一个参与者确定用例。首先，要将获取到的需求分配给与其相关的参与者，以便可以针对每个参与者进行工作，而无遗漏；其次，进行合并操作。在合并之前，要明确为什么要合并，知道了合并的目的，才可能选择正确的合并操作。合并后，将产生用例。将识别到的参与者和合并生成的用例，通过用例图的形式整理出来，以获得用例模型的框架，如图 11-9 所示。

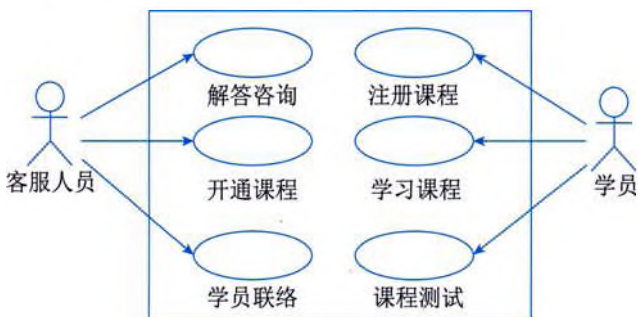


图 11-9 用例图示例

在确定用例的过程中，需要注意以下问题：

(1) 用例命名。用例的命名应该注意采用“动词（短语）+ 名词（短语）”的形式，例如，“开通课程”和“课程测试”等。而且，最好能够对用例进行编号，这也是实现需求跟踪管理的重要技巧，通过编号可以将用户的需求落实到特定的用例中去。

(2) 不能混淆用例和用例所包含的步骤。例如，“开通课程”功能要经过验证学员信息、检查学员权限、保存开通记录、修改课程选修人数等步骤才能完成，在系统中这些步骤不能作为单独的功能对外提供，它们只是一个用例所包含的事件流，或是用例的子功能。

(3) 注意区分业务用例和系统用例。当针对整个业务领域建模时，需要使用业务用例，其

中会涉及大量的人工活动,例如,在线教育平台系统中有一项重要工作是“编写教材”,这就是业务用例,是信息系统无法完成的。信息系统作为整个业务系统的一部分,只负责实现系统的部分功能,因此,只需要识别出系统用例,而不需考虑业务用例。

4. 细化用例描述

用例建模的主要工作是书写用例规约(Use Case Specification),而不是画图。用例模板为一个给定项目的所有人员定义了用例规约的结果,其内容至少包括用例名、参与者、目标、前置条件、事件流(基本事件流和扩展事件流)和后置条件等,其他的还可以包括非功能需求和用例优先级等。

用例描述通常包括以下几个部分:

(1) 用例名称。用例名称应该与用例图相符,并写上其相应的编号。

(2) 简要说明。对用例为参与者所传递的价值结果进行描述,应注意语言简要,使用用户能够阅读的自然语言。

(3) 事件流。事件流是指当参与者和系统试图达到一个目标时所发生的一系列活动,也就是用例所完成的工作步骤。在编写时应注意使用简单的语法,主语明确,语义易于理解;明确写出“谁控制球”,也就是在事件流描述中,让读者直观地了解是参与者在控制还是系统在控制;从俯视的角度来编写,指出参与者的动作,以及系统的响应;描述用户意图和系统职责,而不叙述具体的行为和技术细节,特别是有关用户界面的细节。

执行一个用例的事件流有多种可能的路线,其中主事件流(基本事件流)是指能够满足目标的典型的成功路径,主事件流通常不包括任何条件和分支,符合大多数人的期望,从而更容易理解和扩展;备选事件流(扩展事件流)也称为备选路径,是完成用例可能出现失败的情况、分支路径或扩展路径,为了不影响用例活动清晰的主线,将这些分支处理全部抽取出来作为备选事件流。

另外,事件流的编写过程也是可以分阶段迭代进行的,对于优先级高的用例花更多的时间,更加细化;对优先级低的用例可以先简略地将主事件流描述清楚,备选事件流留待以后处理。对于一些事件流较为复杂的,可以在用例描述中引用顺序图、状态图和通信图等手段进行描述。

(4) 非功能需求。因为用例所涉及的非功能需求通常很难在事件流中进行表达,因此单列为一小节进行描述。在非功能需求的描述方面,一定要注意使其可度量和可验证。否则,就容易流于形式,形同摆设。

(5) 前置条件和后置条件。前置条件是执行用例之前必须存在的系统状态,如果前置条件不满足,则用例无法启动;后置条件是用例执行完毕系统可能处于的一组状态。一旦用例成功执行,可能会导致系统内部某些状态的变化。而某些用例也可能没有前置条件或后置条件。如果在当前阶段不容易确定前置条件或后置条件,则可以在以后再细化。

(6) 扩展点。如果包括扩展(或包含)用例,则写出扩展(或包含)用例名,并说明在什么情况下使用。

(7) 优先级。说明用户对该用例的期望值,为以后的开发工作确定先后顺序。可以采用满意度/不满意度指标进行说明,例如,设置为1~5的数值。

表 11-2 是图 11-9 中“开通课程”的用例描述，这些内容不一定要一次完成。如果需要调整用例模型，则在调整用例模型之后，还可以修改和细化用例描述。

表 11-2 用例描述示例

1. 用例名称

开通课程（UC02）。

2. 简要说明

为用户开通学习课程的权限，将其标记为“学员”，同时修改所选修课程的选修人数。

3. 事件流

3.1 主事件流

- (1) 客服人员向系统发出“开通课程”请求。
- (2) 系统要求客服人员选择开通课程的类型（软考、考研、专业课程、自学考试、Java 课程）。
- (3) 客服人员做出选择后，系统显示相应界面，让客服人员输入信息，并自动根据权限规则生成权限。
- (4) 客服人员输入学员的相关信息，包括学员用户名、所交费用、交费时间、选修课程名称。
- (5) 系统确认学员所交费用和所选修课程的规定费用一致。
- (6) 系统将所输入的信息存储建档，开通学员课程权限。

3.2 备选事件流

- (1) 如果学员所交费用少于所选修课程的规定费用，则显示所选修课程的规定费用，并要求客服人员选择修改或取消输入。
 - 客服人员选择取消输入，则结束用例，不做存储建档工作。
 - 客服人员选择修改用户所交费用后，转到（5）。
- (2) 如果学员所交费用多于所选修课程的规定费用，则显示多余的费用数量，并要求客服人员选择是转换为学习币还是退还给学员。
 - 客服人员选择转换为学习币，则把多余的费用转换为学习币，记入学员账户中，转到（5）。
 - 客服人员选择退还给学员，转到（5）。
- (3) 如果学员所选修的课程超出了系统规定的选修人数，则提示客服人员，结束用例。

4. 非功能需求

无特殊要求。

5. 前置条件

客服人员登录在线教育平台系统。

6. 后置条件

修改学员权限，修改课程选修人数。

7. 扩展点

无。

8. 优先级

最高（满意度 5，不满意度 5）。

5. 调整用例模型

在建立了初步的用例模型后，还可以利用用例之间的关系来调整用例模型。用例之间的关系主要有包含、扩展和泛化，利用这些关系，把一些公共的信息抽取出来，以便于复用，使得用例模型更易于维护。

(1) 包含关系。当可以从两个或两个以上的用例中提取公共行为时, 应该使用包含关系来表示它们。其中这个提取出来的公共用例称为抽象用例, 而把原始用例称为基本用例或基础用例。

(2) 扩展关系。如果一个用例明显地混合了两种或两种以上的不同场景, 即根据情况可能发生多种分支, 则可以将这个用例分为一个基本用例和一个或多个扩展用例, 这样使描述可能更加清晰。

(3) 泛化关系。当多个用例共同拥有一种类似的结构和行为的时候, 可以将它们的共性抽象成为父用例, 其他的用例作为泛化关系中的子用例。在用例的泛化关系中, 子用例是父用例的一种特殊形式, 子用例继承了父用例所有的结构、行为和关系。

在实际工作中, 系统分析师要谨慎选用这些关系。从上面的介绍可以看出, 包含、扩展和泛化关系都会增加用例的个数, 从而增加用例模型的复杂度。另外, 一般都是在用例模型完成之后才对它进行调整, 在用例模型建立之初不必急于抽象用例之间的关系。

11.5.2 分析模型

分析模型描述系统的基本逻辑结构, 展示对象和类如何组成系统(静态模型), 以及它们如何保持通信, 实现系统行为(动态模型)。

建立分析模型的过程大致包括定义概念类、确定类之间的关系、为类添加职责、建立交互图等, 其中有学者将前三个步骤统称为类-责任-协作者(Class-Responsibility-Collaborator, CRC)建模。

1. 定义概念类

OOA 的中心任务就是要找到系统中的对象或类, 这些类将反映到系统设计中的软件类和系统实现中某个 OOP 语言声明的类。例如, 在领域模型中, 学员学习某门课程是一个事件, 该事件记录了某个学员和某门课程在一定时期内的责任关系, 表达的是领域概念; 在系统设计模型中, 学习记录就是一个软件类。虽然它们是不同的事物, 但领域模型中的命名启发了后者的命名和定义, 从而缩小了表示的差距。在整个系统开发过程中, 要尽量使这些类或对象在不同阶段保持相同的名称。

发现类的方法有很多种, 其中应用最广泛的是名词短语法。它的主要规则是先识别有关问题域文本描述中的名词或名词短语, 然后将它们作为候选的概念类或属性, 其具体步骤如下:

- (1) 阅读和理解需求文档或用例描述。
- (2) 筛选出名词或名词短语, 建立初始类清单(候选类)。
- (3) 将候选类分成三类, 分别是显而易见的类、明显无意义的类和不确定类别的类。
- (4) 舍弃明显无意义的类。
- (5) 小组讨论不确定类别的类, 直到将它们都合并或调整到其他两个类别, 并进行相应的操作。

例如, 根据表 11-2 所描述的“开通课程”用例的事件流, 可以获得候选概念类的清单, 如表 11-3 所示。

表 11-3 候选概念类清单

名词类别	概念类列表
显而易见的类	课程、学习币、学员
明显无意义的类	请求、界面、信息
不确定类别的类	学员账户、课程类型、权限、用户名、费用、课程名称

经过简单分析可以得出，“学员账户”“用户名”“权限”可以归结到“学员”类，作为“学员”类的属性；“课程类型”和“课程名称”可以归结到“课程”类，作为“课程”类的属性；“费用”可以单独列为一个类，称为“费用清单”。这样，针对“开通课程”这个用例，就确定了4个类，分别是课程、学习币、学员和费用清单。

另外，也可以根据描述中的名词类别来发现候选类，例如，划分为人员、组织、物品、设备、事件、规格说明、业务规则或政策等。要注意的是，不是所有的名词或名词短语都是系统中的一个合适的候选类，因为有的在系统之外，有的与系统不相关，有的名词概念较小，只适合于作为某个候选类的属性。因此，必须对其进行一番筛选，把不合适的过滤掉。

2. 确定类之间的关系

当完成了类的寻找工作之后，就需要理清这些类之间的关系。类之间的主要关系有关联、依赖、泛化、聚合、组合和实现等，它们在UML中的表示方式如图11-10所示。

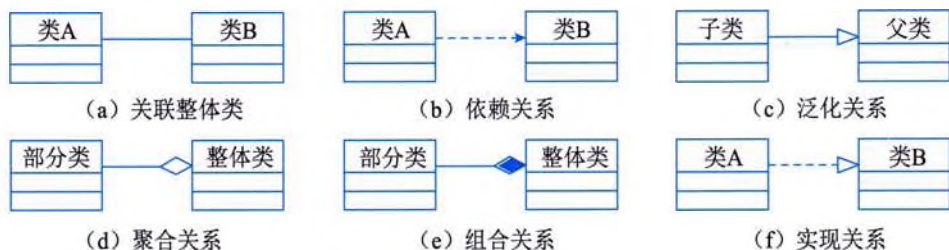


图 11-10 类之间的关系表示

(1) 关联关系。关联提供了不同类的对象之间的结构关系，它在一段时间内将多个类的实例连接在一起。关联体现的是对象实例之间的关系，而不表示两个类之间的关系。其余的关系涉及类元自身的描述，而不是它们的实例。对于关联关系的描述，可以使用关联名称、角色、多重性和导向性来说明，如图11-11所示。

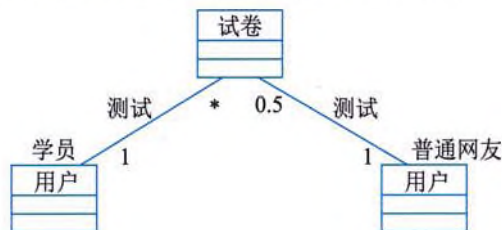


图 11-11 关联关系的表示

角色名称。多重性指定所在类可以实例化的对象数量（重数），即该类的多少个对象在一段特定

关联名称反映该关系的目，并且应该是一个动词，例如，图11-11中的“测试”。如果某种关联的含义对于开发人员和用户都是非常明确的，则可以省略关联名称。关联路径的两端为角色，角色规定了类在关联中所起的作用，例如，图11-11中的“学员”和“普通网友”。一般情况下，只有在关联名称不能明确表述时，才使用

的时间内可以与另一个类的一个对象相关联。例如，图 11-11 中的数字和“*”都表示关联，其中“*”等价于“0..*”。导向性表示可以通过关联从源类导向到目标类，也就是说，给定关联一端的对象就能够容易并直接地得到另一端的对象。导向性用一个箭头表示，如果没有箭头，就认为是一个双向关联或是一个未定义的关联。关联关系的表示方式，同样也适合其他关系的表示，只是箭线符号不同而已。

(2) 依赖关系。两个类 A 和 B，如果 B 的变化可能会引起 A 的变化，则称类 A 依赖于类 B。依赖可以由各种原因引起，例如，一个类向另一个类发送消息、一个类是另一个类的数据成员、一个类是另一个类的某个操作参数等。

(3) 泛化关系。泛化关系描述了一般事物与该事物中的特殊种类之间的关系，也就是父类与子类之间的关系。继承关系是泛化关系的反关系，也就是说，子类继承了父类，而父类则是子类的泛化。

(4) 共享聚集。共享聚集关系通常简称为聚合关系，它表示类之间的整体与部分的关系，其含义是“部分”可能同时属于多个“整体”，“部分”与“整体”的生命周期可以不相同。例如，汽车和车轮就是聚合关系，车子坏了，车轮还可以用；车轮坏了，可以再换一个。

(5) 组合聚集。组合聚集关系通常简称为组合关系，它也是表示类之间的整体与部分的关系。与聚合关系的区别在于，组合关系中的“部分”只能属于一个“整体”，“部分”与“整体”的生命周期相同，“部分”随着“整体”的创建而创建，也随着“整体”的消亡而消亡。例如，一个公司包含多个部门，它们之间的关系就是组合关系。公司一旦倒闭，也就无所谓部门了。

(6) 实现关系。实现关系将说明和实现联系起来。接口是对行为而非实现的说明，而类中则包含了实现的结构。一个或多个类可以实现一个接口，而每个类分别实现接口中的操作。

确定了类之间的关系之后，通过 UML 的类图将这些关系记录下来，形成领域模型。例如，图 11-12 就是与在线教育平台系统相关的一个简单的领域模型。

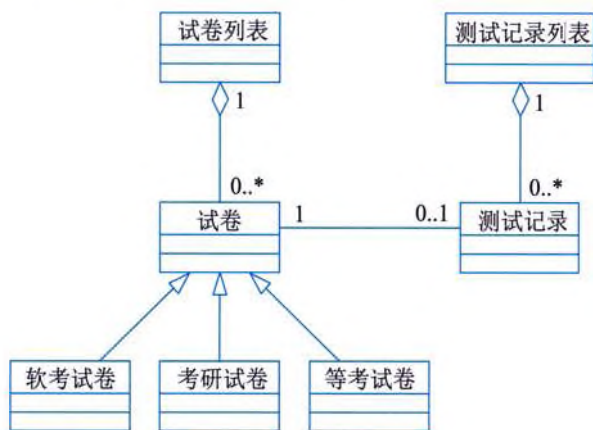


图 11-12 领域模型示例

3. 为类添加职责

找到了反映问题域本质的主要概念类，而且理清了它们之间的协作关系之后，系统分析师

就可以为这些类添加其相应的职责。类的职责包括两个方面的内容，一个是类所维护的知识，即成员变量或属性；另一个是类能够执行的行为，即成员方法或责任。

属性是描述类静态特征的一个数据项。系统分析师可以与用户进行交谈，提出问题来帮助寻找类的属性。从概念建模的角度来看，属性越简单越好。要保持属性的简单性，应该做到只定义与系统责任和系统目标有关的属性；使用简单数据类型来定义属性，不使用可由其他属性导出的属性（冗余属性）；不为类关联定义属性。最后，要对属性加以说明，包括名称、解释和数据类型，以及其他的一些要求。

对于类的责任的确定，可以根据用例描述中的动词来进行判断，然后再进行筛选。这个过程与类的识别过程是类似的，此处不再赘述。系统分析师应该通用性地描述类的成员方法，例如，“交费”和“组卷”等。另外，根据封装性原则，信息和与其相关的行为应该存在于同一个类中。关于一个事物的信息应该包含在单个类中，而不是分布在多个类中。但是，在适当的时候，可以在相关的类之间分享责任。

要注意的是，为类添加职责与找到类之间的关系一样，这个阶段也只能找到那些主要的、明显的、与业务规则相关的部分。切忌在这个阶段不断地细化，甚至引入一些与具体实现相关的技术内容（例如，数据库和分布式对象等）。

4. 建立交互图

多个对象的行为通常采用对象交互来表示，UML 2.0 提供的交互图有顺序图、交互概览图、通信图和定时图。每种图出于不同视点对行为有不同的表现能力，其中最常用的是顺序图，几乎可以用在任何系统的场合。顺序图的基本元素有对象、参与者、生命线、激活框、消息和消息路线，其中消息是顺序图的灵魂。当对象行为复杂并存在多种不同状态转换时，还要用到反映对象状态变化的状态图。

5. 分析模型的详细程度问题

对于分析模型的详细程度，各种文献说法不一，在实践中的做法也不一样。有些文献建议只列出类以及类之间的主要关系，不要对关系进行描述，更不要体现类的职责；而有些文献则认为应该将这些东西都列出来。其实，干巴巴地讨论这个问题是没有任何意义的。在整个系统开发的过程中，分析模型是不断演变的，应随着开发的深入加入新的内容，或修改已有内容，逐渐完善，演变完整。最初的分析模型主要是围绕着领域知识进行的，对现实的事物进行建模。而后，则不断地加入设计的元素，最终演变成为运行于计算机上的系统。

总之，模型不是要开发的目标产物，而是开发过程中的一个辅助工作，只要能够利用其帮助团队更好地开发，详细也罢，简约也罢，都是好模型。

11.6 需求定义

系统分析师在获取了用户的需求，并进行了详细分析之后，接下来的工作就需要把这些软件需求形成文档，作为系统后续开发的基础，这就是需求定义。

11.6.1 需求定义的方法

需求定义的过程也就是形成需求规格说明书的过程，通常有两种需求定义的方法，分别是严格定义方法和原型方法。

1. 严格定义方法

严格定义也称为预先定义，需求的严格定义建立在以下的基本假设之上：

(1) 所有需求都能够被预先定义。假设意味着，在没有实际系统运行经验的情况下，全部的系统需求均可通过逻辑推断得到。这对某些规模较小、功能简单的系统是可能的，但对那些功能庞大、复杂且较大的系统显然是困难的。即使事先做了深入细致的调查和分析，当用户见到新系统的实际效果时，也往往会改变原先的看法，会提出修改或更进一步增加系统功能的要求，因此，再好的预先定义技术也会经常反复修改。这是因为人们对新事物的认识与理解将随着直观、实践的过程进一步加深，这是与人类认识世界的客观规律相一致的。所以，能够预先定义出所有需求的假设在许多场合是不能成立的。

(2) 开发人员与用户之间能够准确而清晰地交流。假设认为，用户与开发人员之间，虽然每个人都有自己的专业、观点和行话，但在系统开发过程中可以使用图形（或文本）等通信工具进行交流，进行清晰且有效的沟通，这种沟通是必不可少的。遗憾的是，在实际开发中，往往对一些共同的约定，每个人可能都会产生自己的理解和解释。即使采用结构化语言、判定树和判定表等工具，仍然存在不精确和技术上的不严密。这将导致人们有意无意地带有个人的不同理解而各行其是，因此，在多学科、多行业人员之间进行有效的通信和交流是有一定困难的。

(3) 采用图形（或文字）可以充分体现最终系统。在使用严格定义需求的开发过程中，开发人员与用户之间交流与通信的主要工具是定义报告，包括叙述文字、图形、逻辑规则和数据字典等技术工具。它们都是静止的、被动的，不能实际演示，很难在用户头脑中形成一个具体的形象。因此，要用静止的图形或文字描述来体现一个动态的系统是比较困难的。

由此可见，严格定义法的基本假设在许多情况下并不成立，传统的结构化方法面临着一些难以跨越的障碍。为此，需要探求一种变通的方法。

2. 原型方法

原型方法以一种与严格定义法截然不同的观点看待需求定义问题。原型化的需求定义过程是一个开发人员与用户通力合作的反复过程。从一个能满足用户基本需求的原型系统开始，允许用户在开发过程中提出更好的要求，根据用户的要求不断地对系统进行完善，它实质上是一种迭代的循环型的开发方式。采用原型方法时需要注意以下几个问题：

(1) 并非所有的需求都能在系统开发前被准确地说明。事实上，要想严密、准确地定义任何事情都是有一定难度的，更不用说是定义一个庞大系统的全部需求。用户虽然可以叙述他们所需最终系统的目标和大致功能，但是对某些细节问题却往往不可能十分清楚。一个系统的开发过程，无论对于开发人员还是用户来说，都是一个学习和实践的过程，为了帮助他们在这个过程中提出更完善的需求，最好的方法就是提供现实世界的实例——原型，对原型进行研究和实践，并进行评价。

(2) 项目干系人之间通常都存在交流上的困难，原型提供了克服该困难的一个手段。用户和开发人员通过屏幕和键盘进行对话、讨论和交流，从他们自身的理解出发来测试原型。原型系统由于直观性和动态性，而使项目干系人之间交流上的困难得到较好的克服。

(3) 需要实际的、可供用户参与的系统模型。虽然图形和文字描述是一种较好的通信交流工具，但是，其最大缺陷是缺乏直观的和感性的特征，因此不易理解对象的全部含义。交互式的系统原型能够提供生动的需求规格说明，用户见到的是一个“活”的和实际运行着的系统。实际使用在计算机上运行的系统，显然比理解纸面上的系统要深刻得多。

(4) 有合适的系统开发环境。随着计算机硬件、软件技术和软件工具的迅速发展，软件的设计与实现工作越来越方便，对系统进行局部性修改甚至重新开发的代价大大降低。因此，对大系统的原型化已经成为可能。

(5) 反复是完全需要和值得提倡的，需求一旦确定，就应遵从严格的方法。系统分析师应该鼓励用户改进他们的系统，只有做必要的改变后，才能使用户和系统间获得更加良好的匹配。所以，从某种意义上说，严格定义需求的方法实际上抑制了用户在需求定义以后再改进的要求，这对提高最终系统的质量是有害的。另一方面，原型方法的使用并不排除严格定义方法的运用，当通过原型在演示中得到明确的需求定义后，应采用行之有效的严格方法来完成最终系统的开发。

11.6.2 软件需求规格说明书

软件需求规格说明书（Software Requirement Specification, SRS）是需求开发活动的产物，编制该文档的目的是使项目干系人与开发团队对系统的初始规定有一个共同的理解，使之成为整个开发工作的基础。SRS 是软件开发过程中最重要的文档之一，对于任何规模和性质的软件项目都不应该缺少。

1. SRS 的编写方法

通常有三种方法编写 SRS，分别列举如下：

(1) 用好的结构化和自然语言编写文本型文档。

(2) 建立图形化模型，这些模型可以描述转换过程、系统状态及其变化、数据关系、逻辑流、对象类及其关系。

(3) 编写形式化规格说明，这可以通过使用数学上精确的形式化逻辑语言来定义需求。

尽管形式化规格说明具有很强的严密性和精确度，但由于其所使用的形式化语言只有极少数专业人员才熟悉，所以，这一方法一直没有在实际应用中得到普遍使用。虽然文本型文档具有许多缺点，但在大多数软件工程中，它仍是编写 SRS 最现实的方法。包含了功能和非功能需求的基于文本的 SRS 已经为大多数项目所接受。图形化模型通过提供另一种需求视图，增强了 SRS，一般作为文本型文档的补充或附加描述功能。

不管采用什么方法编写 SRS，都应注意其正确性、完整性、一致性、必要性、可行性、确定性、可修改性和可追踪性。在工作实践中，为了能够让非技术人员更好地阅读和理解，应该尽可能通过自然语言和简单的图表来表达，以防止造成不必要的误会。

2. SRS 的内容和格式

在国家标准《计算机软件文档编制规范》(GB/T 8567—2006)中,提供了一个 SRS 的文档模板和编写指南,其中规定 SRS 应该包括以下几部分内容:

(1) 范围。本部分包括 SRS 适用的系统和软件的完整标识,(若适用)包括标识号、标题、缩略词语、版本号和发行号;简述 SRS 适用的系统和软件的用途,描述系统和软件的一般特性;概述系统开发、运行和维护的历史;标识项目的投资方、需方、用户、承建方和支持机构;标识当前和计划的运行现场;列出其他有关的文档;概述 SRS 的用途和内容,并描述与其使用有关的保密性和私密性的要求;说明编写 SRS 所依据的基线。

(2) 引用文件。列出 SRS 中引用的所有文档的编号、标题、修订版本和日期,还应标识不能通过正常的供货渠道获得的所有文档的来源。

(3) 需求。这一部分是 SRS 的主体部分,详细描述软件需求,可以分为以下项目:所需的状态和方式、需求概述、需求规格、软件配置项能力需求、软件配置项外部接口需求、软件配置项内部接口需求、适应性需求、保密性和私密性需求、软件配置项环境需求、计算机资源需求(包括硬件需求、硬件资源利用需求、软件需求和通信需求)、软件质量因素、设计和实现约束、数据、操作、故障处理、算法说明、有关人员需求、有关培训需求、有关后勤需求、包装需求和其他需求,以及需求的优先次序和关键程度。

(4) 合格性规定。这一部分定义一组合格性的方法,对于(3)中的每个需求,指定所使用的方法,以确保需求得到满足。合格性方法包括演示、测试、分析、审查和特殊的合格性方法(例如,专用工具、技术、过程、设施和验收限制等)。

(5) 需求可追踪性。这一部分包括从 SRS 中每个软件配置项的需求到其涉及的系统(或子系统)需求的双向可追踪性。

(6) 尚未解决的问题。如果有必要,可以在这一部分说明软件需求中的尚未解决的遗留问题。

(7) 注解。这一部分包含有助于理解 SRS 的一般信息,例如,背景信息、词汇表、原理等。这一部分应包含为理解 SRS 需要的术语和定义,所有缩略语和它们在 SRS 中的含义的字母序列表。

(8) 附录。提供那些为便于维护 SRS 而单独编排的信息(例如,图表、分类数据等)。为便于处理,附录可以单独装订成册,按字母顺序编排。

11.7 需求验证

资深软件工程师都知道,当以 SRS 为基础进行后续开发工作,如果在开发后期或在交付系统之后才发现需求存在问题,这时修补需求错误就需要做大量的工作。相对而言,在系统分析阶段,检测 SRS 中的错误所采取的任何措施都将节省相当多的时间和资金。因此,有必要对于 SRS 的正确性进行验证,以确保需求符合良好特征。需求验证也称为需求确认,其活动是为了确定以下几个方面的内容:

(1) SRS 正确地描述了预期的、满足项目干系人需求的系统行为和特征。

- (2) SRS 中的软件需求是从系统需求、业务规格和其他来源中正确推导而来的。
- (3) 需求是完整的和高质量的。
- (4) 需求的表示在所有地方都是一致的。
- (5) 需求为继续进行系统设计、实现和测试提供了足够的基础。

11.7.1 需求评审

在软件开发的每个阶段结束前，都需要进行技术评审。所谓技术评审，是指对工作产品进行检查以发现产品中所存在的问题，其中的工作产品也称为工件，它不一定是最终的系统，也可以是一个文档、一个原型或一段代码等。例如，需求评审就是需求开发阶段结束前进行的技术评审，此时的产品就是 SRS。SRS 的评审是一项精益求精的技术，它可以发现那些二义性的或不确定性的需求，为项目干系人提供在需求问题上达成共识的方法。

1. 技术评审的类型

根据 IEEE 的词汇表，技术评审可以分为以下三种类型：

- (1) 评审。评审是指一次正式的会议，在会议上向用户或其他项目干系人介绍一个或一组工作产品，以征求对方的意见和批准。
- (2) 检查。检查是一种正式的评估方法，将由非制作者本人的个人或小组详细检查工作产品，以验证是否有错误、是否违反开发标准、是否存在其他问题。
- (3) 走查。走查是一个评审过程，由某个开发人员领导一个或多个开发团队成员对其工作产品进行检查，由其他成员针对技术、风格、可能的错误、是否违反开发标准和是否存在其他问题提出意见。

在实际工作中，技术评审可以分为正式评审和非正式评审。正式评审是指通过召开评审会的形式，组织多个专家，将工作产品涉及的人员集合在一起，并定义好评审人员的角色和职责，对工作产品进行正规的会议评审。而非正式评审并没有这种严格的组织形式，一般也不需要将人员集合在一起评审，而是通过电子邮件、文件汇签，甚至是网络聊天等多种形式对工作产品进行评审。

非正式评审对于获得分散而随机的反馈是有效的，但非正式评审是非系统化的、不彻底的，它在实施过程中具有不一致性。非正式评审可以根据个人爱好的方式进行评审，而正式评审则需要遵循预先定义好的一系列步骤和过程。正式评审的内容需要记录在案，包括确定材料，评审小组对工作产品是否完整或者是否需要进一步工作的判定，以及对所发现的错误和所提出的问题的总结。

2. 如何做好需求评审

需求评审是需求开发阶段的最后工作，需求评审一旦通过，将进入系统设计阶段。如果在系统设计甚至后续阶段再跨里程碑来修改需求，所花费的代价将大大增加。

在如何做好需求评审工作方面，业内人士总结了一些经验，简单列举如下：

- (1) 分层次评审。用户的需求是分层次的，对不同层次的需求，其描述形式是有区别的，参与评审的人员也是不同的。

(2) 正式评审与非正式评审相结合。正式评审与非正式评审各有利弊,但往往非正式评审比正式评审的效率更高,更容易发现问题。因此,在评审时,应该更灵活地利用这两种方式。

(3) 分阶段评审。应该在需求形成的过程中进行分阶段的评审,而不是在需求最终形成后再进行评审。分阶段评审可以将原本需要进行的大规模评审拆分成各个小规模评审,降低了需求返工的风险,提高了评审的质量。

(4) 精心挑选评审人员。需求评审可能涉及的人员包括建设单位的中高层管理人员和操作人员、IT主管和采购主管,承建单位的市场人员、系统分析师、系统架构设计师、软件工程师、测试人员、质量保证人员、实施人员和项目经理,以及第三方的领域专家等。为了保证评审的质量和效率,需要精心挑选评审人员。首先,要保证使不同类型的人员都要参与进来。否则,很可能会漏掉重要的需求;其次,在不同类型的人员中要选择那些真正和系统相关的,对系统有足够了解的人员参与进来。否则,很可能使评审的效率降低或者最终不切实际地修改了系统需求。

(5) 对评审人员进行培训。在很多情况下,评审人员是领域专家而不是进行评审活动的专家,他们没有掌握评审的方法、技巧和过程等。因此,需要对他们进行培训,以便于评审人员能够紧紧围绕评审的目标来进行,能够控制评审活动的节奏,提高评审效率。

(6) 充分利用需求评审检查单。需求评审检查单可以帮助评审人员系统而全面地发现需求中的问题。需求评审检查单可以分为两类:需求形式的检查单和需求内容的检查单。需求形式的检查可以由质量保证人员负责,主要检查SRS的格式是否符合质量标准;需求内容的检查是由评审人员负责的,主要检查需求内容是否达到了系统目标、是否有遗漏、是否有错误等,这是需求评审的重点。

(7) 建立标准的评审流程。需求评审需要建立正规的流程,按照流程中定义的活动进行规范的评审过程。例如,在评审流程定义中可能规定评审的进入条件、评审需要提交的资料、每次评审会议的人员职责分配、评审的具体步骤和评审通过的条件等。

(8) 做好评审后的跟踪工作。在需求评审后,需要根据评审人员提出的问题进行评价,以确定哪些问题是必须纠正的,哪些可以不纠正,并给出充分、客观的理由和证据。当确定需要纠正的问题后,要形成书面的需求变更申请,进入需求变更的管理流程,并确保变更的执行,在变更完成后,要进行复审。切忌评审完毕后,没有对问题进行跟踪,而无法保证评审结果的落实,使前期的评审问题不能妥善解决。

(9) 充分准备评审。评审质量的好坏很大程度上取决于在评审会议前的准备活动。通常出现的问题是,SRS在评审会议前并没有提前下发给评审人员,没有留出更多、更充分的时间让评审人员阅读需求文档。更有甚者,没有执行需求评审的进入条件,在评审文档中存在大量、低级的错误,或者没有在评审前进行沟通,文档中存在方向性的错误,从而导致评审的效率很低,质量很差。对评审的准备工作,也应当定义一个检查单,在评审之前对照检查单落实每项准备工作。

11.7.2 需求测试

在许多项目中,软件测试是后期的开发活动。与需求相关的问题总是依附在软件产品中,直到通过系统测试或经用户运行系统后才可能最终发现它们。而事实上,软件测试应该从需求定义开始,如果在开发过程的早期就开始制订测试计划和进行测试用例的设计,就可以在发生

错误时立即检测到并纠正它。这样，就可以防止这些错误进一步“放大”，并且可以减少测试和维护费用。

另一方面，需求的遗漏和错误具有很强的隐蔽性，仅仅通过阅读 SRS，通常很难想象在特定环境下的系统行为。只有在业务需求基本明确，用户需求部分确定时，同步进行需求验证，才可能及早发现问题，从而在需求开发阶段以较低的代价解决这些问题。

1. 概念测试用例

实际上，需求开发阶段不可能有真正意义上的测试进行，因为还没有可执行的系统，需求测试仅仅是基于文本需求进行“概念”上的测试。然而，以功能需求为基础（SA 方法）或者从用例派生出来（OO 方法）的测试用例，可以使项目干系人更清楚地了解系统的行为。虽然没有在系统上执行测试用例，但是涉及测试用例的简单动作可以解释需求的许多问题。这种测试用例通常称为概念测试用例，即不是真正执行的测试用例，它们可以发现 SRS 中的错误、二义性和遗漏，还可以进行模型分析，以及作为用户验收测试的基础。在正式的系统测试中，还可以将它们细化成测试用例。

概念测试用例的设计应该覆盖用例的主事件流和备选事件流（OO 方法），或者系统的功能描述（SA 方法），以及在需求获取和分析期间所确定的约束条件。通常意义上，概念测试用例来源于用户需求，重点反映用例（或功能需求条目）的描述，完全独立于实现，仅仅是概念上的描述测试脚本。例如，图 11-9 中的“开通课程”用例的概念测试用例可以按表 11-4 的方式进行设计。

表 11-4 概念测试用例设计示例

-
- | |
|--|
| <p>(1) 客服人员向系统发出“开通课程”请求，系统检查客服人员是否登录及是否具有相应的权限。如果没有登录系统或权限不够，则提示客服人员进行相应操作，并结束用例；否则，执行第(2)步。</p> <p>(2) 系统显示课程类型列表，包括软考、考研、专业课程、自学考试、Java 课程。要求客服人员选择开通课程的类型，然后单击“确定”按钮。</p> <p>(3) 客服人员做出选择后，系统显示信息输入界面，让客服人员输入信息。</p> <p>(4) 客服人员输入学员的相关信息，包括学员用户名、所交费用、交费时间、选修课程名称。在输入的过程中，系统要检查客服人员的输入是否符合相关规则：“用户名”必须是系统中存在的用户名，“所交费用”必须为整数，“交费时间”必须在当前时间之前，“选修课程名称”必须是“课程”中已有的课程。</p> <p>(5) 如果学员所交费用多于所选修课程的规定费用，则显示多余的费用数量，并要求客服人员选择是转换为学习币还是退还给学员。如果客服人员选择“转换为学习币”，则把多余的费用转换为学习币，记入学员账户中，继续执行；如果客服人员选择“退还给学员”，则继续执行。</p> <p>(6) 如果学员所交费用少于所选修课程的规定费用，则显示所选修课程的规定费用，并要求客服人员选择修改或取消输入。如果客服人员选择“取消输入”，则结束用例；否则，客服人员修改用户所交费用后继续执行。</p> <p>(7) 如果学员所选修的课程超出了系统规定的选修人数，则提示客服人员，结束用例。</p> <p>(8) 修改学员权限，使学员权限为所选修课程对应的权限。</p> <p>(9) 修改选修课程的人数，使学员所选修课程的人数加 1，结束用例。</p> |
|--|
-

设计概念测试用例的另一种方法是将表 11-4 中的文字叙述通过图形的方式描述出来，形成对话图（Dialog Map），模仿用户和系统的对话。这种方式更加直观，但不适合描述约束条件，

也不适合描述流程比较复杂的用例。

2. 需求测试的过程

基于概念测试用例进行需求测试的基本过程如下：

(1) 需求测试人员根据概念测试用例所描述的若干可能的过程，进行“概念上”的执行，期望发现遗漏的、错误的和不必要的需求。

(2) 根据测试结果快速修改对应的需求文档，完成一轮完整的需求测试过程。

基于该过程，需求测试人员应用概念测试用例来进行需求测试，直至概念测试覆盖所有的用例和功能需求条目为止。需求测试人员和系统分析师根据需求测试的结果，进一步讨论修订 SRS 的内容和版本。至此，整个需求测试过程结束。

在实际工作中，可以将需求测试人员作为测试人员中的特殊类型来培养，使他们能够对需求是否正确进行检查。当然，需求测试人员也可以是有经验的系统分析师，但是，他必须脱离开发部门，加入测试部门，这样才能保证测试不是“既当运动员，又当裁判员”，以确保需求测试的效果。

另外，对于系统的功能需求，也可以用 RAD 工具建立界面原型，用户通过原型的操作来确定需求是否与期望相同。对于那些不合理的需求，需求测试人员要能够分辨得出来，并与用户进行核对，以确定用户的真实需求。从这个角度来看，可以说需求测试是由需求测试人员和用户共同来执行的。

11.8 需求管理

需求管理包括需求文档的追踪管理、变更控制、版本控制等管理性活动。软件需求开发的最终文档经过评审批准后，就定义了开发工作的需求基线（Baseline）。

11.8.1 需求变更

在当前的软件开发过程中，需求变更已经成为一种常态。需求变更的原因有很多种，可能是需求获取不完整，存在遗漏的需求；可能是对需求的理解产生了误差；也可能是业务变化导致了需求的变化等。一些需求的改进是合理的而且不可避免，要使软件需求完全不变更，基本上是不可能的。但毫无控制的变更会导致项目陷入混乱，不能按进度完成或者软件质量无法保证。

软件需求文档应该精确描述要交付的产品，这是一个基本的原则。为了使开发组织能够严格控制软件项目，应该确保以下事项：

- (1) 仔细评估已建议的变更。
- (2) 挑选合适的人选对变更做出判定。
- (3) 变更应及时通知所有相关人员。
- (4) 项目要按一定的程序来采纳需求变更，对变更的过程和状态进行控制。

1. 变更控制过程

变更控制过程用来跟踪已建议变更的状态，使已建议的变更确保不会丢失或疏忽。一旦确

定了需求基线，应该使所有已建议的变更都遵循变更控制过程。

(1) 问题分析和变更描述：当提出一份变更提议后，需要对该提议做进一步的问题分析，检查它的有效性，从而产生一个更明确的需求变更提议。

(2) 变更分析和成本计算：当接受该变更提议后，需要对需求变更提议进行影响分析和评估。变更成本计算应该包括对该变更所引起的所有改动的成本，例如修改需求文档、相应的设计、实现等工作成本。一旦分析完成并且被确认，应该进行是否执行这一变更的决策。

(3) 变更实现：当确定执行该变更后，需要根据该变更的影响范围，按照开发的过程模型执行相应的变更。在计划驱动过程模型中，往往需要回溯到需求分析阶段开始，重新作对应的需求分析、设计和实现等步骤；在敏捷开发模型中，往往会将需求变更纳入到下一次迭代的执行过程中。

变更控制过程并不是给变更设置障碍。相反地，它是一个渠道和过滤器，通过它可以确保采纳最合适的变更，使变更产生的负面影响降到最低。控制需求变更与项目其他配置的管理决策也有着密切的联系。项目管理应该达成一个策略，用来描述如何处理需求变更，而且策略应具有现实可行性。

常见的需求变更策略：

- (1) 所有需求变更必须遵循变更控制过程。
- (2) 对于未获得批准的变更，不应该做设计和实现工作。
- (3) 变更应该由项目变更控制委员会决定实现哪些变更。
- (4) 项目风险承担者应该能够了解变更的内容。
- (5) 绝不能从项目配置库中删除或者修改变更请求的原始文档。
- (6) 每一个集成的需求变更必须能跟踪到一个经核准的变更请求，以保持水平可追踪性。

目前存在很多需求变更跟踪工具，这些工具用来收集、存储和管理需求变更。问题跟踪工具也可以随时按变更状态分类报告变更请求的数目。

2. 变更控制委员会

变更控制委员会（Change Control Board, CCB）是项目所有者权益代表，负责裁定接受哪些变更。CCB 由项目所涉及的多方成员共同组成，通常包括用户和实施方的决策人员。CCB 是决策机构，不是作业机构，通常 CCB 的工作是通过评审手段来决定项目是否能变更，但不提出变更方案。

变更控制委员会可能包括如下方面的代表。

- (1) 产品或计划管理部门。
- (2) 项目管理部门。
- (3) 开发部门。
- (4) 测试或质量保证部门。
- (5) 市场部或客户代表。
- (6) 制作用户文档的部门。
- (7) 技术支持部门。

(8) 帮助桌面或用户支持热线部门。

(9) 配置管理部门。

变更控制委员会应该有一个总则,用于描述变更控制委员会的目的、授权范围、成员构成、做出决策的过程及操作步骤。总则也应该说明举行会议的频度和事由。管理范围描述该委员会能做什么样的决策,以及有哪一类决策应上报到高一级的委员会。过程及操作步骤如下:

(1) 制定决策。制定决策过程的描述应确认:

①变更控制委员会必须到会的人数或做出有效决定必须出席的人数。

②决策的方法(例如投票、一致通过或其他机制)。

③变更控制委员会主席是否可以否决该集体的决定。

变更控制委员会应该对每个变更权衡利弊后做出决定。“利”包括节省的资金或额外的收入、增强的客户满意度、竞争优势、减少上市时间;“弊”是指接受变更后产生的负面影响,包括增加的开发费用、推迟的交付日期、产品质量的下降、减少的功能、用户不满意度。

(2) 交流情况。一旦变更控制委员会做出决策,指派的人员应及时更新请求的状态。

(3) 重新协商约定。变更总是有代价的,即使拒绝的变更也会因为决策行为(提交、评估、决策)而耗费资源。当项目接受了重要的需求变更时,为了适应变更情况要与管理部和客户重新协商约定。协商的内容可能包括推迟交货时间、要求增加人手、推迟实现尚未实现的较低优先级的需求,或者质量上进行折中。

11.8.2 需求跟踪

需求跟踪包括编制每个需求同系统元素之间的联系文档,这些元素包括其他需求、体系结构、其他设计部件、源代码模块、测试、帮助文件和文档等,这是为了在整个项目的工件之间形成水平可追踪性。跟踪能力信息使变更影响分析十分便利,有利于确认和评估实现某个建议的需求变更所必需的工作。

需求跟踪提供了由需求到产品实现整个过程范围的明确查阅能力。需求跟踪的目的是建立与维护“需求-设计-编程-测试”之间的一致性,确保所有的工作成果符合用户需求。

需求跟踪有两种方式:

(1) 正向跟踪。检查产品需求规格说明书中的每个需求是否都能在后继工作成果中找到对应点。

(2) 逆向跟踪。检查设计文档、代码、测试用例等工作成果是否都能在产品需求规格说明书中找到出处。

正向跟踪和逆向跟踪合称为双向跟踪。不论采用何种跟踪方式,都要建立与维护需求跟踪矩阵(即表格)。需求跟踪矩阵保存了需求与后继工作成果的对应关系。

跟踪能力是优秀需求规格说明书的一个特征,为了实现可跟踪能力,必须统一地标识出每一个需求,以便能明确地进行查阅。

需求跟踪是个要求手工操作且劳动强度很大的任务,要求组织提供支持。随着系统开发的进行和维护的执行,要保持关联信息与实际一致。跟踪能力信息一旦过时,可能再也不会重建它。在实际项目中,往往采用专门的配置管理工具来实现需求跟踪。